CONFIDENTIAL | CODE AUDIT

Pet2U — Code Review Report

Full-Stack Laravel E-Commerce Application

Date **26 March 2026**

Framework **Laravel 12 (PHP)**

Stack **Blade · Bootstrap 5 · Vite**

Scope **Full Codebase Audit**

TABLE OF CONTENTS

1. Executive Summary

Pet2U is a feature-rich, full-stack Laravel e-commerce application for a pet pharmacy. It supports both over-the-counter (OTC) and prescription-only medicine (POM) orders, a multi-tier admin system, payment processing via Super Payments, Royal Mail shipping, Google Shopping merchant sync, and a full blog/SEO system.

The codebase demonstrates a good understanding of Laravel patterns — repository pattern, DTOs, service classes, middleware, and role-based access control are all present and meaningful. However, the audit identified **5 security vulnerabilities**, several medium-priority code quality issues, and a number of minor improvements. Five issues were fixed immediately during this audit and a detailed roadmap for the remaining items is provided below.

2. Application Overview

Property	Detail
Framework	Laravel 12
PHP	8.2+
Frontend	Blade Templates, Bootstrap 5.2, Tailwind CSS 4, Vite 6
Database	SQLite (dev) / MySQL or PostgreSQL (production)
Auth	Laravel built-in + Socialite (Google/Facebook OAuth), Email Verification, Spatie Roles
Payment	Super Payments gateway (HMAC-signed webhooks)
Shipping	Royal Mail API integration
Roles	admin, dispensary, pharmacist, user, seo_manager, seo_team
Migrations	48 migration files
Controllers	43 controllers
Models	34 Eloquent models
Repositories	30 repository classes
Key Features	POM prescription uploads, dynamic pet questions, order approval workflow, sitemap generation, Google Merchant sync, blog, chatify messaging, multi-language

3. Overall Code Quality Scores

4 / 10
SECURITY

6 / 10
CODE QUALITY

6 / 10
ARCHITECTURE

1 / 10	7 / 10	8 / 10
TEST COVERAGE	DATABASE DESIGN	FEATURE COMPLETENESS

Overall Rating: 5.3 / 10 — Needs Work Before Production.

The application is functionally rich and uses the right patterns in the right places. The primary concerns are the near-total absence of automated tests, several security vulnerabilities, and some large monolithic classes that will become hard to maintain as the product grows. The scoring will improve significantly once the critical security issues and test coverage gaps are addressed.

4. Security Findings — Critical & High Priority

CRITICAL **Missing Refund Webhook Signature Validation** ✓ Fixed

`app/Http/Controllers/RefundWebhookController.php` — line 32

The refund webhook endpoint accepted any POST request without verifying it was genuinely from Super Payments. The code had a comment `// TODO: Implement webhook signature validation` but the body was empty. An attacker could have forged refund status updates, potentially marking orders as refunded without any real payment being returned.

Fixed:
Implemented full HMAC-SHA256 + base64 signature verification matching the pattern already used in SuperPaymentWebhookController. Reads the `super-signature` header, extracts the timestamp and v1 signature, reconstructs the expected hash, and rejects the request with a 401 if it does not match. Any environment without the confirmation ID configured now also hard-fails.

CRITICAL Hardcoded Default Password in Migration ✓ Fixed

`database/migrations/0001_01_01_000000_create_users_table.php` — line 25

The `password` column had `->default('test123')`. Any user record inserted without explicitly providing a password would silently receive the plaintext string "test123" as their hashed or unhashed password. This is a well-known default credential and is trivially exploitable.

Fixed:
Removed the default value entirely. Password is now a required field with no fallback, forcing every code path to provide a proper bcrypt-hashed value.

HIGH Dangerous use of `extract()` — Variable Injection ✓ Fixed

`app/Http/Controllers/DashboardController.php` — 8 occurrences across 4 methods

`extract()` blindly imports all keys from an array into the current variable scope. If the source array ever contained user-controlled data, an attacker could overwrite critical variables such as `$user`, `$dateFilter`, or `$currentMonth`. Even without user input, it makes code very hard to trace since there is no explicit declaration of where each variable comes from.

Fixed:
All 8 `extract()` calls in `adminDashboard`, `dispensaryDashboard`, `userDashboard`, and `seoDashboard` replaced with explicit variable assignments (`$dateRange = $params['dateRange']`, etc.). Every variable now has a clear, traceable origin.

HIGH No Rate Limiting on Public Endpoints ✓ Fixed

`routes/web.php`

Contact form submissions, cart operations, product notification subscriptions, and question file uploads had no rate limiting. This opens the application to spam attacks on the contact form, cart flooding that could exhaust sessions, and abuse of the stock notification system to harvest email data.

Fixed:
Added Laravel `throttle` middleware: contact form POST limited to 10 requests/minute, cart routes 60/minute, product notification subscribe 10/minute, and questions endpoints 30/minute.

HIGH **46 Instances of Unescaped Blade Output (`{!! !!}`)** Requires Manual Review

`resources/views/` — 46 occurrences across frontend and admin views

`{!! $variable !!}` outputs raw HTML without escaping. If any of these variables contain user-generated content (product descriptions, blog posts, user notes, contact messages) that is not sanitised upstream, this is a direct cross-site scripting (XSS) vulnerability. Review each occurrence and either switch to `{{ }}` or ensure the upstream value is passed through an HTML sanitiser (e.g., HTMLPurifier or Laravel's `e()`).

Not auto-fixed
— requires manual review of each instance to determine intent. Some uses (e.g. rendering admin-entered rich text) may be acceptable, others (user input) must be escaped or sanitised.

HIGH **Sensitive Data Logged in Webhook Controllers** Requires Attention

`app/Http/Controllers/RefundWebhookController.php`, `SuperPaymentWebhookController.php`

Both webhook controllers log full request payloads (`$request->all()`) including transaction IDs, amounts, and payment references. Log files can be read by anyone with server access, and logging payment data may violate PCI DSS guidelines.

Recommendation:
Replace `$request->all()` in log calls with an explicit whitelist of safe fields (`eventType` , `transactionStatus`). Never log `transactionId` in full — mask or omit it.

5. Medium Priority Issues

MEDIUM **Monolithic Controllers and Repository**

OrderController: 1,220 lines | **DashboardController**: 1,321 lines | **FrontendController**: 915 lines | **OrderRepository**: 1,757 lines

These classes handle too many responsibilities (SRP violation). They are difficult to test, hard to maintain, and risky to modify since a change in one area can unintentionally break another. OrderRepository alone handles CRUD, checkout, order creation, shipping, payment, and admin management.

MEDIUM **Session-Only Cart (No Database Persistence)**

`app/Http/Controllers/CartController.php` , `app/Repositories/CartRepository.php`

The shopping cart is stored entirely in the PHP session. If a session expires (or the user switches devices), the cart is silently lost. This is inconsistent with the persistent "My Pets" feature and is likely to cause customer complaints and lost conversions.

MEDIUM **Exposed /clear Route (No Auth Guard)**

`routes/web.php` — line 33

The public `/clear` route calls `config:cache` , `cache:clear` , `route:clear` , and `view:clear` without any authentication. Anyone who discovers this URL can repeatedly flush the application cache, causing performance degradation. Critically, it also contains a commented-out `migrate:fresh --seed` — if uncommented accidentally, this would wipe the entire production database.

MEDIUM **Foreign Key Type Mismatch**

`database/migrations/2025_06_27_052622_create_products_table.php`

The products table uses UUID as its primary key, but a foreign key on the `users` table uses `unsignedBigInteger` (the users table uses auto-increment BigInt). These types must match to enforce referential integrity. A type mismatch can prevent the foreign key constraint from being applied by the database engine.

MEDIUM **Inconsistent Cascade Delete Strategy**

`database/migrations/`

Some foreign keys use `->onDelete('cascade')` while others use `->nullOnDelete()` with no apparent logic behind the distinction. This can lead to orphaned records in the database, data integrity issues, and unpredictable behaviour when an order or user is deleted.

MEDIUM **N+1 Query Risk**

`app/Http/Controllers/FrontendController.php` , `OrderController.php`

Several controller methods load related models inside loops rather than using eager loading (`->with()`). On a product listing page or order history with many items, this generates one database query per record. With a large product catalogue, this will cause severe performance degradation.

6. Minor / Low Priority Issues

Issue	Location	Priority
Typo in users migration: "Acout section" instead of "About section"	`migrations/0001_01_01…`	Fixed
Merge conflict markers left in README.md (`<<<<<<< HEAD`)	`README.md`	Fixed
Commented-out `migrate:fresh --seed` sitting in a public route	`routes/web.php:39`	Remove immediately
No database seeders — local development requires manual test data	`database/seeders/`	Low
Brands table `status` field is a string instead of boolean/enum	`migrations/create_brands_table.php`	Low
No API documentation (no OpenAPI/Swagger)	Whole codebase	Low
No .env.example file for deployment configuration guidance	Root	Low
Timestamps not consistently cast across all models	`app/Models/`	Minor
Inline `@php` blocks with business logic inside Blade views	`resources/views/`	Minor
Uses `env()` directly in service code instead of `config()`	`SuperPaymentWebhookController.php`	Minor

7. What Was Fixed in This Audit

#	Issue	File Changed	Severity
1	Implemented missing refund webhook HMAC signature validation (was a TODO stub)	`RefundWebhookController.php`	Critical
2	Removed hardcoded `default('test123')` from password column	`migrations/create_users_table.php`	Critical
3	Replaced all 8 `extract()` calls with explicit variable assignments	`DashboardController.php`	High
4	Added rate limiting throttle middleware to 4 public endpoint groups	`routes/web.php`	High
5	Resolved unmerged git conflict markers in README	`README.md`	Low
6	Fixed typo "Acout" → "About" in users migration default string	`migrations/create_users_table.php`	Minor

8. Code Strengths

✅ **Repository Pattern** — 30 repositories provide clean data abstraction throughout the app.

✅ **DTO Pattern** — 24 Data Transfer Objects enforce type-safe data passing between layers.

✅ **Service Classes** — Payment, shipping, sitemap, and merchant sync are cleanly separated into services.

✅ **Role-Based Access** — Spatie/permission integration provides a well-structured RBAC system.

✅ **Webhook Security (Payments)** — SuperPaymentWebhookController has solid signature verification that was the model for the refund fix.

✅ **Custom Middleware** — 10+ middleware including POM validation, XSS protection, and OTC guest checkout.

✅ **Transactional Emails** — Order confirmation, POM approval/rejection, and stock alert emails are implemented.

✅ **SEO Features** — XML sitemap generation, robots.txt, and schema.org structured data are built in.

✅ **Multi-language** — LaravelLocalization is fully integrated for multi-locale support.

✅ **UUIDs on Orders/Products** — Using UUIDs on key tables prevents ID enumeration attacks.

✅ **Error Handling** — Try-catch blocks with structured logging are used consistently throughout controllers.

✅ **POM Workflow** — The prescription-only medicine approval workflow is a sophisticated domain-specific feature.

9. Recommendations & Next Steps

The following items are prioritised by impact. The first three should be addressed before any production launch.

1 **Review and fix all 46 unescaped `{!! !!}` outputs in Blade views**

Grep for `{!!` across `resources/views/` and for each occurrence, confirm the variable is either admin-only content or passes through an HTML sanitiser. Switch to `{{ }}` anywhere user-supplied content could appear.

2 **Secure or remove the public `/clear` route**

Move it behind an `auth` + `admin` middleware guard, or better, use a protected artisan command via Forge/Envoyer. Delete the commented-out `migrate:fresh --seed` line permanently.

3 **Write feature tests for critical paths**

There are currently zero automated tests. At minimum, write feature tests for: place order, POM approval workflow, payment webhook handling, and refund webhook handling. These paths handle money and patient medication — they must be regression-tested.

4 **Mask sensitive data in logs**

Replace all `Log::info('...', $request->all())` in webhook controllers with an explicit allow-list of safe fields. Never log full transaction IDs or amounts to disk.

5 **Persist the cart to the database**

Create a `carts` table linked to `user_id` (nullable for guests, using a session token). Migrate session cart data to DB on login. This prevents lost carts and enables remarketing.

6 **Break down monolithic controllers and OrderRepository**

Extract checkout logic into a `CheckoutService`, refund logic into a `RefundService`, and shipping into a dedicated service. Each controller method should be under 40 lines.

7 **Fix foreign key type mismatch on products table**

Ensure the `user_id` foreign key on `products` matches the primary key type on `users` (`unsignedBigInteger`). Create a migration to correct the column type if needed.

8 **Add N+1 query protection**

Enable Laravel's `Model::preventLazyLoading()` in the dev environment. Fix all resulting exceptions with eager loading (`->with([...])`) before deploying to production.

9 **Replace `env()` with `config()` in application code**

Calling `env()` directly in controllers/services means the value is not cached when `config:cache` is run in production. Move all env reads into config files and reference them via `config('key')`.

10 **Add database seeders and a development .env.example**

Create seeders for users with each role, sample products, and categories so any developer can spin up a working dev environment with `php artisan migrate:fresh --seed`. Update `.env.example` to document all required environment variables including Super Payments credentials.